CAREERTIQ
Cloud DevOps Engineering Series | Unit 4: Infrastructure Automation

HANDS-ON LAB REPORT

Chapter 2: Prerequisites & AWS Environment Setup

Installing AWS CLI, Terraform and Ansible: Enterprise Smart Library Management Platform

Student Name	**Shubham Deshmukh**
PRN	2125UDSM1027
Department	AI and DS
Environment	Windows 11 + WSL2, Ubuntu 22.04 LTS, AWS region us-east-1
Date	11 October 2026
GitHub Repository	https://github.com/shubhamdeshmukh1027/library-platform-devops

Table of Contents

1. Chapter 2 Overview & Objectives 3
Lab environment verification: WSL2 and Ubuntu 22.04 3

2. Section 2.1: Installing AWS CLI, Terraform and Ansible 4
2.1.1 Step 1: Workstation Baseline Utilities 4
2.1.2 Step 2: Installing AWS Command Line Interface (AWS CLI v2) 4
2.1.3 Step 3: Installing HashiCorp Terraform CLI 5
2.1.4 Step 4: Installing Ansible Core & Dependencies 6

3. Section 2.2: AWS Programmatic Authentication & STS Verification 6
IAM user and access key creation (AWS Console) 6
3.1 Configuring AWS Credentials via CLI 8
3.2 Verifying AWS Security Token Service (STS) 8

4. Section 2.3: SSH Key Pair Generation & EC2 Import 8
4.1 Generating 4096-Bit RSA SSH Key Pair 9
4.2 Importing Public Key into AWS EC2 Key Pairs 9

5. Section 2.4: Toolchain Verification Matrix & Sanity Checks 10

6. Troubleshooting Common Environment Setup Errors 10

7. Project Repository on GitHub (Proof of Work) 11

8. Conclusion 13

1. Chapter 2 Overview & Objectives

Curriculum Module	Target Platform	Focus Tools	Release Status
Unit 4: Infrastructure Automation	Ubuntu 22.04 LTS / macOS / WSL2	AWS CLI v2, Terraform, Ansible	v1.0.0 Verified Lab Guide

Before executing Infrastructure as Code (IaC) or Configuration Management automation, engineers must establish a standardized, deterministic local workstation environment. This guide provides exact, production-verified instructions to install, configure, authenticate and validate the three foundational tools required for the Enterprise Smart Library Management Platform:

- **AWS Command Line Interface (AWS CLI v2):** The unified tool to manage AWS cloud services, query resources and authenticate programmatic API sessions.
- **HashiCorp Terraform (v1.7+):** The declarative Infrastructure as Code engine used to provision cloud fabric (VPCs, Subnets, Security Groups, EC2 compute and S3 storage).
- **Ansible Core (v2.16+):** The agentless configuration management engine executing playbooks over OpenSSH to prepare servers, install Docker and Nginx, and launch the application.

LAB PREREQUISITES & SYSTEM REQUIREMENTS

Operating System: Linux (Ubuntu 20.04/22.04 LTS recommended), Debian 11/12, macOS (Apple Silicon or Intel), or Windows 10/11 running WSL2 Ubuntu.

Hardware: Minimum 2 CPU cores, 4 GB RAM, 15 GB free disk space, and outbound internet connectivity on ports 80, 443 and 22.

Lab environment verification: WSL2 and Ubuntu 22.04

The lab was executed on Windows 11 using WSL2 with the Ubuntu 22.04 LTS distribution.

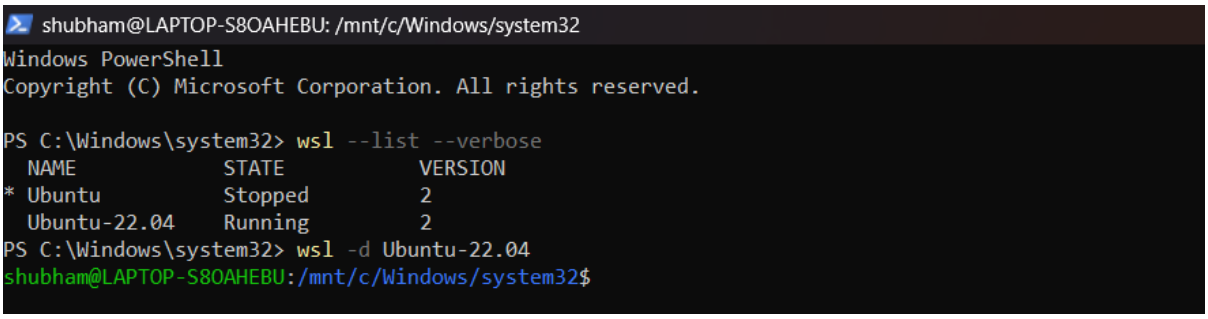


Figure 1: WSL distributions: Ubuntu-22.04 running on VERSION 2

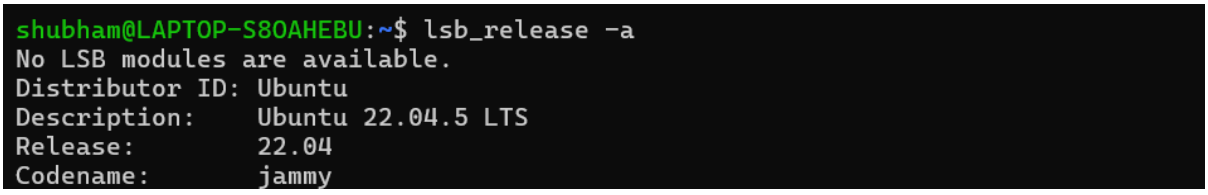


Figure 2: Ubuntu 22.04.5 LTS (codename jammy) confirmed with lsb_release -a

2. Section 2.1: Installing AWS CLI, Terraform and Ansible

2.1.1 Step 1: Workstation Baseline Utilities

Ensure local package repositories are refreshed and the necessary archive utilities, compilers and security certificates are installed.

Terminal: Updating Repositories and Installing Baseline Tools

```
# Update local package indexes and upgrade outdated packages
sudo apt-get update && sudo apt-get upgrade -y

# Install core system utilities, cURL, unzip tools and Python prerequisites
sudo apt-get install -y curl wget unzip jq git software-properties-common \
  ca-certificates gnupg apt-transport-https python3 python3-pip python3-venv
```

```
shubham@LAPTOP-S8OAHEBU:~$ sudo apt-get update
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 https://apt.releases.hashicorp.com jammy InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:5 https://ppa.launchpadcontent.net/ansible/ansible/ubuntu jammy InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
shubham@LAPTOP-S8OAHEBU:~$
```

Figure 3: apt-get update: package lists refreshed, HashiCorp and Ansible repositories reachable

```
shubham@LAPTOP-S8OAHEBU:~$ curl --version | head -1; git --version; jq --version; unzip -v | head -1; python3 --version
curl 7.81.0 (x86_64-pc-linux-gnu) libcurl/7.81.0 OpenSSL/3.0.2 zlib/1.2.11 brotli/1.0.9 zstd/1.4.8 libidn2/2.3.2 libpsl/0.21.0 (+libidn2/2.3.2) libssh/0.9.6
/openssl/zlib nghttp2/1.43.0 librtmp/2.3 OpenLDAP/2.5.20
git version 2.34.1
jq-1.6
UnZip 6.00 of 20 April 2009, by Debian. Original by Info-ZIP.
Python 3.10.12
shubham@LAPTOP-S8OAHEBU:~$
```

Figure 4: Baseline tools verified: curl 7.81.0, git 2.34.1, jq 1.6, unzip 6.00, Python 3.10.12

2.1.2 Step 2: Installing AWS Command Line Interface (AWS CLI v2)

AWS CLI version 2 is the official modern binary distribution for managing Amazon Web Services. Unlike version 1, it is distributed as a self-contained bundle that does not rely on host Python environments.

Terminal: Downloading and Installing AWS CLI v2 on Linux (x86_64)

```
# 1. Download official AWS CLI v2 installer package
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"

# 2. Extract the downloaded archive
unzip -q awscliv2.zip

# 3. Execute installation script (creates binaries in /usr/local/bin)
sudo ./aws/install --update

# 4. Clean up installation artifacts
rm -rf aws awscliv2.zip

# 5. Verify installation
aws --version
```

```
shubham@LAPTOP-S8OAHEBU:~$ which aws; aws --version
/usr/local/bin/aws
aws-cli/2.37.12 Python/3.14.6 Linux/6.18.33.2-microsoft-standard-WSL2 exe/x86_64.ubuntu.22
shubham@LAPTOP-S8OAHEBU:~$
```

Figure 5: AWS CLI v2 installed at /usr/local/bin/aws (version 2.37.12)

> **Note for macOS & Windows WSL2 Users**
> **macOS (Homebrew):** run brew install awscli terraform ansible.
> **macOS (Official PKG):** download AWSCLIV2.pkg from the official AWS documentation.
> **Windows WSL2:** follow the Linux terminal commands above inside your Ubuntu WSL2 bash shell.

2.1.3 Step 3: Installing HashiCorp Terraform CLI

HashiCorp signs all package releases using official GPG keys. The official APT repository is registered securely and the verified Terraform binary is installed.

Terminal: Registering HashiCorp GPG Key and Installing Terraform

```
# 1. Add HashiCorp official GPG signing key to secure keyrings
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor \
  -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

# 2. Add HashiCorp official Debian/Ubuntu repository source list
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
  https://apt.releases.hashicorp.com $(lsb_release -cs) main" \
  | sudo tee /etc/apt/sources.list.d/hashicorp.list

# 3. Update APT package index to include the HashiCorp repository
sudo apt-get update

# 4. Install Terraform CLI
sudo apt-get install -y terraform

# 5. Enable optional shell autocomplete for Bash / Zsh
terraform -install-autocomplete

# 6. Verify Terraform installation
terraform -version
```

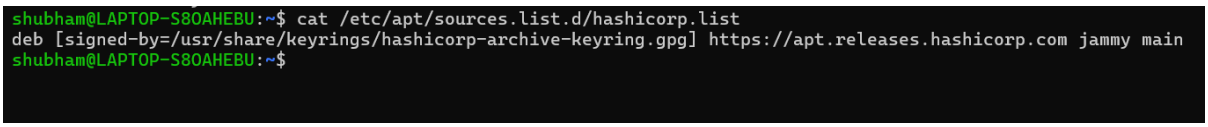


Figure 6: HashiCorp repository registered for Ubuntu jammy

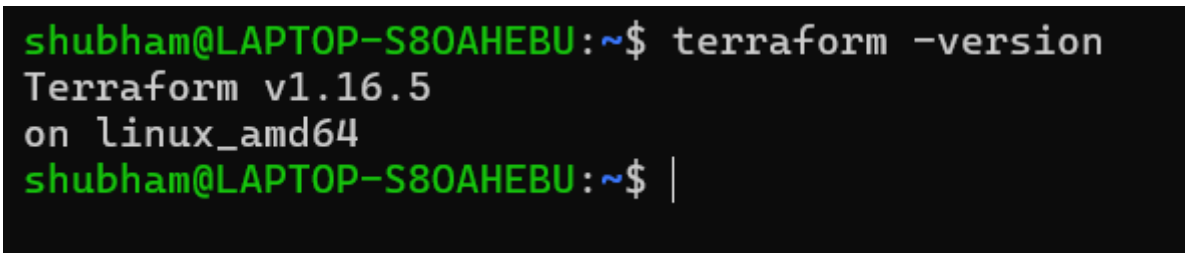


Figure 7: Terraform v1.16.5 installed (requirement: v1.7 or higher)

2.1.4 Step 4: Installing Ansible Core & Dependencies

Ansible requires Python 3.10+ on the control node. On Ubuntu, installing from the official Ansible Personal Package Archive (PPA) ensures access to the latest stable release of Ansible Core and built-in modules.

Terminal: Adding Ansible PPA and Installing Ansible Core

```
# 1. Add Ansible official PPA repository
sudo add-apt-repository --yes --update ppa:ansible/ansible

# 2. Install Ansible Core package
sudo apt-get install -y ansible

# 3. Install community and AWS cloud collection extensions
ansible-galaxy collection install amazon.aws community.general

# 4. Verify Ansible installation and Python runtime path
ansible --version
```

```
shubham@LAPTOP-S8OAHEBU:~$ ansible --version
ansible [core 2.17.14]
  config file = /etc/ansible/ansible.cfg
  configured module search path = ['/home/shubham/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/shubham/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.10.12 (main, Aug 31 2026, 10:18:17) [GCC 11.4.0] (/usr/bin/python3)
  jinja version = 3.0.3
  libyaml = True
shubham@LAPTOP-S8OAHEBU:~$
```

Figure 8: Ansible core 2.17.14 running on Python 3.10.12

```
shubham@LAPTOP-S8OAHEBU:~$ ansible-galaxy collection list | grep -E "amazon.aws|community.general"
amazon.aws                                8.2.1
community.general                         9.5.2
shubham@LAPTOP-S8OAHEBU:~$
```

Figure 9: Collections installed: amazon.aws 8.2.1 and community.general 9.5.2

3. Section 2.2: AWS Programmatic Authentication & STS Verification

To permit Terraform and the AWS CLI to provision infrastructure, programmatic credentials must be configured using an IAM user with appropriate permissions (for example EC2, VPC, S3 and Security Group policies).

IAM user and access key creation (AWS Console)

A dedicated IAM user named devops-lab-user was created (IAM > Users > Create user) with the AdministratorAccess policy attached, and an access key was generated for CLI use. The access key identifier and account number are redacted in the screenshots.

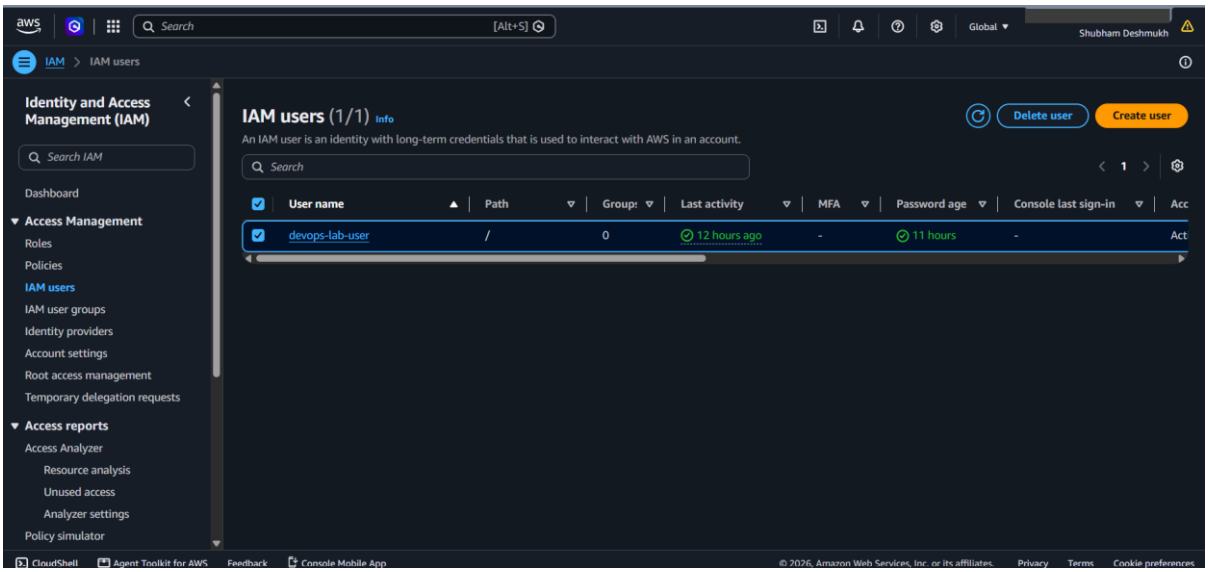


Figure 10: IAM > Users: the devops-lab-user IAM user

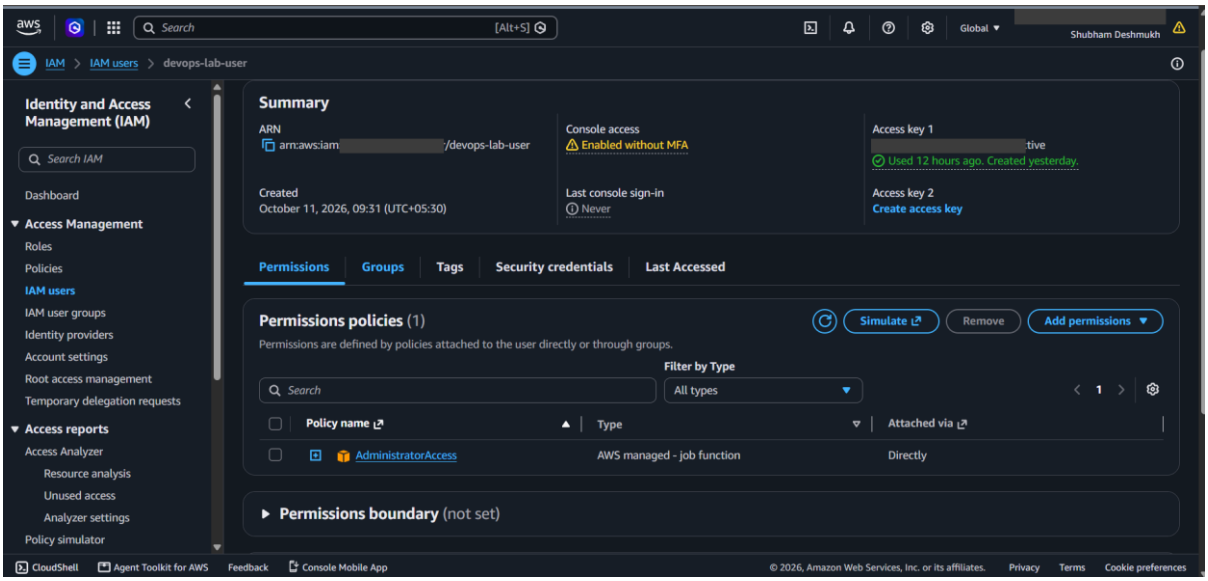


Figure 11: devops-lab-user with the AdministratorAccess policy attached

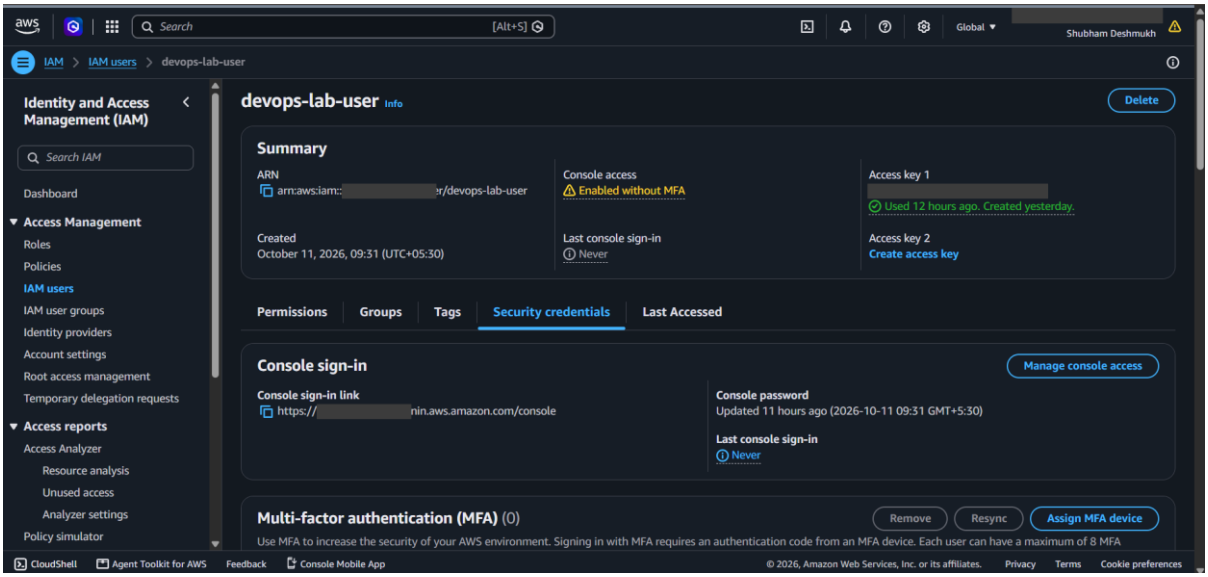


Figure 12: Security credentials tab: Access key 1 is Active (identifier redacted)

3.1 Configuring AWS Credentials via CLI

Terminal: Running 'aws configure' Interactive Setup

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-east-1
Default output format [None]: json
```

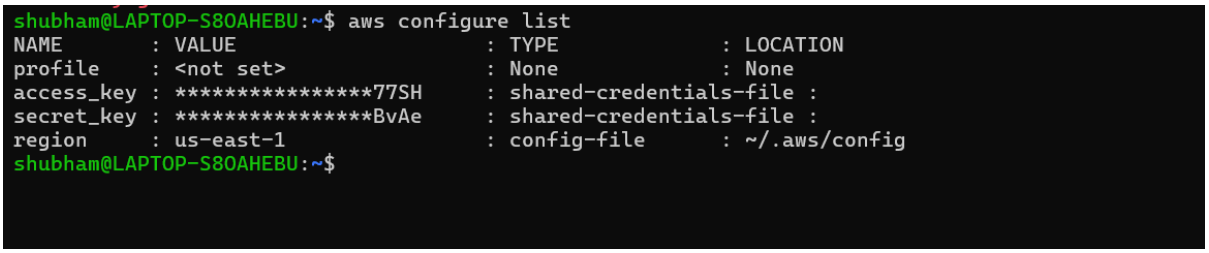


Figure 13: aws configure list: keys masked, region us-east-1 saved in ~/.aws/config

3.2 Verifying AWS Security Token Service (STS)

Verify that the credentials are functioning and identify the authenticated IAM principal.

Terminal: Verifying Authenticated IAM Identity

```
aws sts get-caller-identity
```

Expected Terminal Output: JSON Caller Identity

```
{
    "UserId": "AIDAS6SAMPLEUSEREXAMPLE",
    "Account": "123456789012",
    "Arn": "arn:aws:iam::123456789012:user/devops-trainer-admin"
}
```

Actual output from the lab

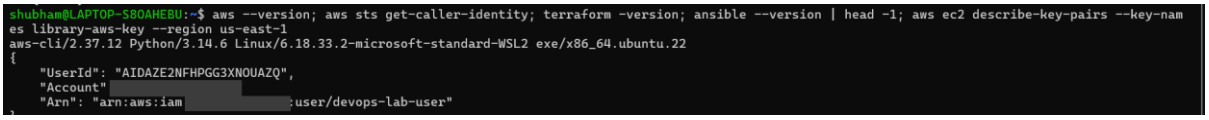


Figure 14: STS confirms the caller is IAM user devops-lab-user (account number redacted)

> **SECURITY GOVERNANCE NOTE**
> Never hardcode AWS Access Keys into Git repositories or Terraform .tf files. The AWS CLI stores credentials securely in ~/.aws/credentials and ~/.aws/config, which Terraform automatically detects and uses at runtime.

4. Section 2.3: SSH Key Pair Generation & EC2 Import

Ansible operates agentlessly over OpenSSH. To let Ansible authenticate securely with the provisioned EC2 instance, a 4096-bit RSA key pair is generated on the local machine and the public key is imported into AWS.

4.1 Generating 4096-Bit RSA SSH Key Pair

Terminal: Generating Key Pair with Strict File Permissions

```
# 1. Create secure SSH directory if not present
mkdir -p ~/.ssh

# 2. Generate a 4096-bit RSA key without passphrase for automated CI/CD execution
ssh-keygen -t rsa -b 4096 -f ~/.ssh/library-aws-key -N "" -C "devops@careertiq-library"

# 3. Restrict private key permissions to read-only for owner (mandatory for OpenSSH)
chmod 400 ~/.ssh/library-aws-key
chmod 644 ~/.ssh/library-aws-key.pub

# 4. Confirm key generation
ls -la ~/.ssh/library-aws-key*
```

```
shubham@LAPTOP-S8OAHEBU:~$ ls -la ~/.ssh/library-aws-key*
-r-------- 1 shubham shubham 3389 Oct 10 22:25 /home/shubham/.ssh/library-aws-key
-rw-r--r-- 1 shubham shubham  750 Oct 10 22:25 /home/shubham/.ssh/library-aws-key.pub
shubham@LAPTOP-S8OAHEBU:~$
```

Figure 15: Private key is read-only for the owner (-r--------), public key is 644

4.2 Importing Public Key into AWS EC2 Key Pairs

Terminal: Registering Public Key in Region us-east-1

```
aws ec2 import-key-pair --key-name "library-aws-key" \
  --public-key-material fileb://~/.ssh/library-aws-key.pub \
  --region us-east-1
```

Expected Output: AWS EC2 Key Pair Confirmation

```
{
    "KeyFingerprint": "b4:37:38:12:44:98:ef:5a:7b:19:30:fa:8c:11:7d:e2",
    "KeyName": "library-aws-key",
    "KeyPairId": "key-0a9b8c7d6e5f41234"
}
```

Actual output from the lab

```
shubham@LAPTOP-S8OAHEBU:~$ aws ec2 describe-key-pairs --key-names library-aws-key --region us-east-1
{
    "KeyPairs": [
        {
            "KeyPairId": "key-04c19c4d592e036b0",
            "KeyType": "rsa",
            "Tags": [],
            "CreateTime": "2026-10-11T05:00:48.218000+00:00",
            "KeyName": "library-aws-key",
            "KeyFingerprint": "f8:d6:c1:48:13:c9:c2:fe:0a:a7:6f:57:1f:e4:99:9f"
        }
    ]
}
shubham@LAPTOP-S8OAHEBU:~$
```

Figure 16: describe-key-pairs: library-aws-key (RSA) exists in us-east-1

To confirm that the key in AWS is the key on the local machine, the MD5 fingerprint of the local public key was computed. It matches the KeyFingerprint returned by AWS (f8:d6:c1:48:13:c9:c2:fe:0a:a7:6f:57:1f:e4:99:9f).

```
shubham@LAPTOP-S8OAHEBU:~$ ssh-keygen -ef ~/.ssh/library-aws-key.pub -m PEM | openssl rsa -RSAPublicKey_in -pubout -outform DER 2>/dev/null | openssl md5 -c
MD5(stdin)= f8:d6:c1:48:13:c9:c2:fe:0a:a7:6f:57:1f:e4:99:9f
shubham@LAPTOP-S8OAHEBU:~$
```

Figure 17: Local public key fingerprint (MD5) matches the AWS KeyFingerprint

5. Section 2.4: Toolchain Verification Matrix & Sanity Checks

Execute the following verification checklist to ensure all tools are functioning correctly before proceeding to Terraform HCL provisioning.

Tool Name	Verification Command	Expected Version / Output	Status
AWS CLI v2	aws --version	aws-cli/2.15.x or higher	**VERIFIED**
AWS Auth	aws sts get-caller-identity	Valid AWS Account ID & IAM User ARN	**VERIFIED**
Terraform CLI	terraform -version	Terraform v1.7.x or higher	**VERIFIED**
Ansible Core	ansible --version	ansible [core 2.16.x or higher]	**VERIFIED**
SSH Key Pair	aws ec2 describe-key-pairs	library-aws-key present in AWS	**VERIFIED**

Actual output from the lab



Figure 18: Final verification: all five checks return the expected output

6. Troubleshooting Common Environment Setup Errors

Symptom / Error Message	Root Cause	Remediation Step
aws: command not found	Binary directory /usr/local/bin is missing from the current user PATH variable.	Run export PATH=$PATH:/usr/local/bin and append it to ~/.bashrc or ~/.zshrc.
Signature verification failed (GPG Error)	System CA certificates are outdated or a firewall is intercepting the HashiCorp GPG key download.	Run sudo apt-get install --reinstall ca-certificates and re-fetch the key via the official URL.
Permissions 0644 for private key are too open	Private SSH key has permissive permissions allowing other users to read it. OpenSSH refuses the connection.	Run chmod 400 ~/.ssh/library-aws-key to restrict access to the file owner.
InvalidAccessKeyId: The AWS Access Key Id does not exist	Typo in the access key, or the IAM user/key was deleted or deactivated in the console.	Generate a fresh access key pair in the AWS IAM Console and re-run aws configure.

NEXT STEP: INFRASTRUCTURE AS CODE PROVISIONING
With Chapter 2 setup complete, the workstation is ready to proceed to **Chapter 3: Terraform Infrastructure as Code Suite** to provision the AWS VPC, Security Group, EC2 compute host and S3 backup bucket.

7. Project Repository on GitHub (Proof of Work)

The lab documentation, screenshots and (in Chapter 3) the Terraform code are stored in a GitHub repository named library-platform-devops. A .gitignore file was created before the first push so that private keys, AWS credentials and Terraform state files are never committed.

```
shubham@LAPTOP-S8OAHEBU:~$ cd ~
git clone https://github.com/shubhamdeshmukh1027/library-platform-devops.git
cd library-platform-devops
Cloning into 'library-platform-devops'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$
```

Figure 19: Repository cloned into ~/library-platform-devops

```
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$ pwd
ls -la
/home/shubham/library-platform-devops
total 16
drwxr-xr-x 3 shubham shubham 4096 Oct 11 11:28 .
drwxr-x--- 9 shubham shubham 4096 Oct 11 11:28 ..
drwxr-xr-x 8 shubham shubham 4096 Oct 11 11:28 .git
-rw-r--r-- 1 shubham shubham  172 Oct 11 11:28 README.md
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$
```

Figure 20: Initial repository contents: .git and README.md

```
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$ git --version; git config --global user.name
git version 2.34.1
shubhamdeshmukh1027
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$
```

Figure 21: Git 2.34.1 installed and user name configured

```
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$ pwd; ls -la; cat .gitignore
/home/shubham/library-platform-devops
total 28
drwxr-xr-x  5 shubham shubham 4096 Oct 11 11:30 .
drwxr-x--- 10 shubham shubham 4096 Oct 11 13:06 ..
drwxr-xr-x  8 shubham shubham 4096 Oct 11 12:55 .git
-rw-r--r--  1 shubham shubham   84 Oct 11 11:30 .gitignore
-rw-r--r--  1 shubham shubham  172 Oct 11 11:28 README.md
drwxr-xr-x  3 shubham shubham 4096 Oct 11 11:30 chapter-2
drwxr-xr-x  3 shubham shubham 4096 Oct 11 11:30 chapter-3
*.pem
*.key
library-aws-key*
.aws/
.ssh/
*.tfstate
*.tfstate.*
.terraform/
*.tfvars
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$
```

Figure 22: .gitignore content and the chapter-2 / chapter-3 folders

```
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$ ls -R chapter-2 chapter-3
chapter-2:
screenshots

chapter-2/screenshots:

chapter-3:
terraform

chapter-3/terraform:
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$
```

Figure 23: Folder structure: chapter-2/screenshots and chapter-3/terraform

```
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$ gh auth status
github.com
  ✓ Logged in to github.com as shubhamdeshmukh1027 (/home/shubham/.config/gh/hosts.yml)
  ✓ Git operations for github.com configured to use https protocol.
  ✓ Token: ********************

shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$
```

Figure 24: GitHub CLI authenticated over HTTPS (token masked)

```
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$ git log --oneline; git remote -v; git status
ca04525 (HEAD -> main, origin/main, origin/HEAD) Add project structure and .gitignore
4573b14 Initial commit
origin  https://github.com/shubhamdeshmukh1027/library-platform-devops.git (fetch)
origin  https://github.com/shubhamdeshmukh1027/library-platform-devops.git (push)
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops$
```

Figure 25: Commit history, remote URL and a clean working tree in sync with origin/main

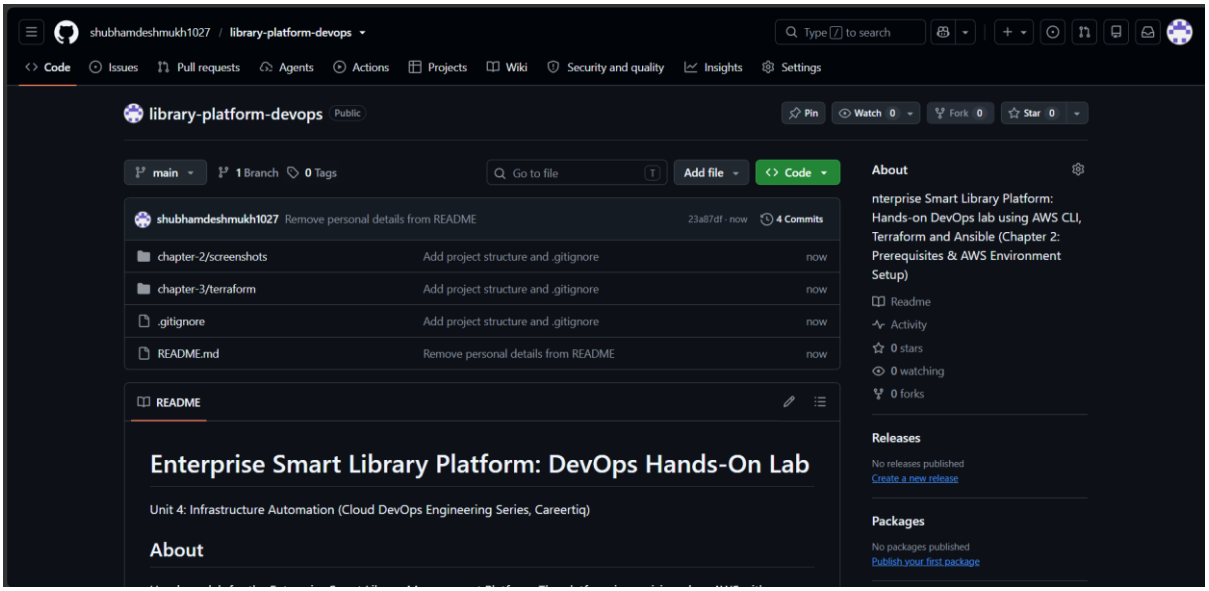


Figure 26: GitHub repository page: folders, .gitignore and README are visible

8. Conclusion

The workstation toolchain (AWS CLI v2, Terraform, Ansible Core, Git and GitHub CLI) was installed and verified on Ubuntu 22.04 under WSL2. An IAM user, AWS CLI credentials and a 4096-bit RSA SSH key pair were set up in us-east-1, the public key fingerprint was matched against AWS, and the project is version controlled on GitHub with a .gitignore that protects secrets. The environment is ready for Chapter 3.

CAREERTIQ
Cloud DevOps Engineering Series | Unit 4: Infrastructure Automation

HANDS-ON LAB REPORT

Chapter 3: Terraform Infrastructure as Code Suite

Provisioning VPC, Subnet, Security Group, EC2 and S3 Backup Bucket: Enterprise Smart Library Management Platform

Student Name	**Shubham Deshmukh**
PRN	2125UDSM1027
Department	AI and DS
Environment	Windows 11 + WSL2, Ubuntu 22.04 LTS, Terraform v1.16.5, AWS region us-east-1
Date	11 October 2026
GitHub Repository	https://github.com/shubhamdeshmukh1027/library-platform-devops

Table of Contents

1. Chapter 3 Overview & Objectives 3
Terraform workflow used in this lab 3

2. Project Structure and Terraform Code 3
2.1 providers.tf 4
2.2 variables.tf 4
2.3 main.tf 4
2.4 outputs.tf 7
2.5 Resources defined by the configuration 7

3. Initializing and Validating 7
3.1 terraform init 7
3.2 terraform fmt and validate 8

4. Plan and Apply 8
4.1 terraform plan 8
4.2 terraform apply 9

5. Verification of the Provisioned Infrastructure 9
5.1 Terraform outputs 9
5.2 EC2 instance 10
5.3 S3 backup bucket 10
5.4 SSH login to the EC2 instance 10

6. Cleanup with terraform destroy 11

7. GitHub Repository Notes 12

8. Troubleshooting Common Terraform Errors 13

9. Conclusion 13

1. Chapter 3 Overview & Objectives

Curriculum Module	Target Platform	Focus Tool	Release Status
Unit 4: Infrastructure Automation	AWS us-east-1 (N. Virginia)	HashiCorp Terraform v1.16.5	Verified Lab Report

In Chapter 2 the workstation was prepared (AWS CLI, Terraform, Ansible, IAM user and SSH key pair). In this chapter Terraform is used as Infrastructure as Code (IaC) to provision the cloud foundation of the Enterprise Smart Library Management Platform. The whole infrastructure is described in code, created with one command and removed with one command, which makes it repeatable and version controlled.

Objectives

- Write Terraform configuration files (providers, variables, main, outputs)
- Initialize, format, validate, plan and apply the configuration
- Provision a VPC, public subnet, Internet Gateway, route table, security group, EC2 instance and S3 backup bucket
- Verify the resources with Terraform outputs, the AWS CLI and an SSH login
- Destroy all resources with terraform destroy to avoid unnecessary AWS charges

Terraform workflow used in this lab

Step	Command	Purpose
1	terraform init	Download providers (aws, random) and prepare the working directory
2	terraform fmt / validate	Format the code and check that the configuration is valid
3	terraform plan	Preview the changes Terraform would make
4	terraform apply	Create or update the real infrastructure
5	terraform output	Show values such as VPC ID, public IP and bucket name
6	terraform destroy	Remove every resource created by the configuration

2. Project Structure and Terraform Code

All code is kept in the chapter-3/terraform folder of the GitHub repository. The ls command shows the four configuration files. The terraform.tfstate file is created by Terraform after the first apply; it stores the real resource IDs and is therefore excluded from Git by .gitignore.

Terminal: Listing the Terraform project files

```
cd ~/library-platform-devops/chapter-3/terraform
ls
```

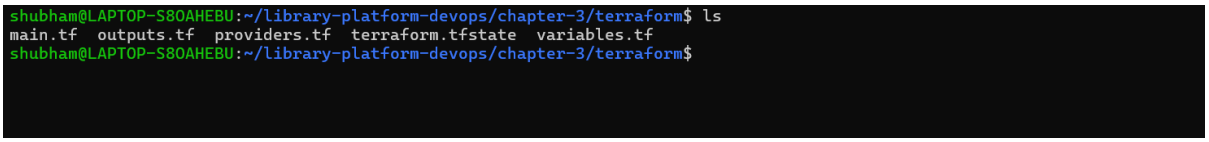


Figure 1: Terraform project files: main.tf, outputs.tf, providers.tf, variables.tf (terraform.tfstate created after apply)

2.1 providers.tf

Declares the required Terraform version, the aws and random providers, and the AWS region.

```
terraform {
  required_version = ">= 1.7.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.0"
    }
    random = {
      source  = "hashicorp/random"
      version = ">= 3.0"
    }
  }
}

provider "aws" {
  region = var.region
}
```

2.2 variables.tf

Input variables make the code reusable: region, instance type, SSH key name (the library-aws-key created in Chapter 2) and the CIDR allowed to SSH.

```
variable "region" {
  type    = string
  default = "us-east-1"
}

variable "instance_type" {
  type    = string
  default = "t3.micro"
}

variable "key_name" {
  type    = string
  default = "library-aws-key"
}

variable "allowed_ssh_cidr" {
  type    = string
  default = "0.0.0.0/0"
}
```

2.3 main.tf

Defines the network, security group, EC2 instance and S3 backup bucket. The Ubuntu 22.04 AMI is looked up automatically with a data source, so no AMI ID is hardcoded.

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags                 = { Name = "library-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "library-igw" }
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "${var.region}a"
  map_public_ip_on_launch = true
  tags                    = { Name = "library-public-subnet" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "library-public-rt" }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

resource "aws_security_group" "web" {
  name        = "library-web-sg"
  description = "Allow SSH and HTTP"
  vpc_id      = aws_vpc.main.id
  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.allowed_ssh_cidr]
  }
  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
```

```
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = { Name = "library-web-sg" }
}

resource "aws_instance" "app" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = var.instance_type
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web.id]
  key_name               = var.key_name
  tags                   = { Name = "library-app-server" }
}

resource "random_id" "suffix" {
  byte_length = 4
}

resource "aws_s3_bucket" "backup" {
  bucket = "library-backup-${random_id.suffix.hex}"
  tags   = { Name = "library-backup" }
}

resource "aws_s3_bucket_versioning" "backup" {
  bucket = aws_s3_bucket.backup.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_public_access_block" "backup" {
  bucket                  = aws_s3_bucket.backup.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

2.4 outputs.tf

```
output "vpc_id" {
  value = aws_vpc.main.id
}

output "instance_public_ip" {
  value = aws_instance.app.public_ip
}

output "s3_bucket_name" {
  value = aws_s3_bucket.backup.bucket
}
```

2.5 Resources defined by the configuration

Terraform resource	Name / purpose
aws_vpc.main	library-vpc, CIDR 10.0.0.0/16
aws_internet_gateway.igw	library-igw, internet access for the VPC

Terraform resource	Name / purpose
aws_subnet.public	library-public-subnet, 10.0.1.0/24 in us-east-1a
aws_route_table.public	library-public-rt, default route 0.0.0.0/0 to the gateway
aws_route_table_association.public	Attaches the route table to the public subnet
aws_security_group.web	library-web-sg, allows SSH (22) and HTTP (80)
aws_instance.app	library-app-server, t3.micro, Ubuntu 22.04, key library-aws-key
random_id.suffix	Random suffix so the S3 bucket name is globally unique
aws_s3_bucket.backup	library-backup-<suffix>, backup bucket
aws_s3_bucket_versioning.backup	Versioning enabled on the bucket
aws_s3_bucket_public_access_block.backup	All public access blocked

3. Initializing and Validating

3.1 terraform init

terraform init prepares the working directory and installs the providers. The output shows hashicorp/aws v6.68.0 and hashicorp/random v3.9.1 in use and the message "Terraform has been successfully initialized!". The versions are recorded in .terraform.lock.hcl.

Terminal: Initializing Terraform

```
terraform init
```

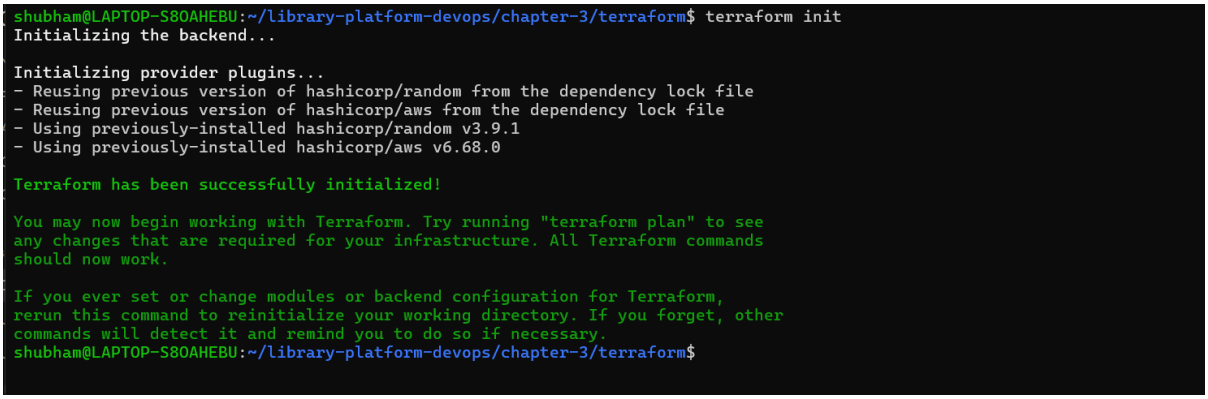


Figure 2: terraform init: providers aws v6.68.0 and random v3.9.1, successfully initialized

3.2 terraform fmt and validate

terraform fmt formats the files to the standard style (no output means nothing needed changing) and terraform validate checks the syntax and references.

Terminal: Formatting and validating the configuration

```
terraform fmt
terraform validate
```

```
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops/chapter-3/terraform$ terraform fmt
terraform validate
Success! The configuration is valid.

shubham@LAPTOP-S8OAHEBU:~/library-platform-devops/chapter-3/terraform$
```

Figure 3: terraform validate: "Success! The configuration is valid."

4. Plan and Apply

4.1 terraform plan

terraform plan compares the configuration with the real infrastructure. All eleven resources already exist in the Terraform state from the first apply, so the plan reports "No changes. Your infrastructure matches the configuration." This shows that Terraform is idempotent: running it again does not create duplicate resources.

Terminal: Previewing changes

```
terraform plan
```

```
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops/chapter-3/terraform$ terraform plan
random_id.suffix: Refreshing state... [id=y2z04A]
data.aws_ami.ubuntu: Reading...
aws_vpc.main: Refreshing state... [id=vpc-02a21dbf08b1f91f4]
aws_s3_bucket.backup: Refreshing state... [id=library-backup-cb6cf4e0]
data.aws_ami.ubuntu: Read complete after 1s [id=ami-062944a84867f2386]
aws_internet_gateway.igw: Refreshing state... [id=igw-02991fdf487a24963]
aws_subnet.public: Refreshing state... [id=subnet-0486e33a97cf76cd9]
aws_security_group.web: Refreshing state... [id=sg-0aeead241b53fa11e]
aws_route_table.public: Refreshing state... [id=rtb-03688b557599529f3]
aws_instance.app: Refreshing state... [id=i-039214559a9c7bbd9]
aws_route_table_association.public: Refreshing state... [id=rtbassoc-0e297e3830573dc27]
aws_s3_bucket_public_access_block.backup: Refreshing state... [id=library-backup-cb6cf4e0]
aws_s3_bucket_versioning.backup: Refreshing state... [id=library-backup-cb6cf4e0]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops/chapter-3/terraform$
```

Figure 4: terraform plan: refreshes state of all resources and reports no changes

4.2 terraform apply

terraform apply creates or updates the infrastructure so that it matches the code. On the first run it asks for confirmation (only "yes" is accepted) and creates the 11 resources. Running it again, as shown below, finishes with 0 added, 0 changed, 0 destroyed and prints the outputs.

Terminal: Applying the configuration

```
terraform apply
```

```
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops/chapter-3/terraform$ terraform apply
random_id.suffix: Refreshing state... [id=y2z04A]
data.aws_ami.ubuntu: Reading...
aws_vpc.main: Refreshing state... [id=vpc-02a21dbf08b1f91f4]
aws_s3_bucket.backup: Refreshing state... [id=library-backup-cb6cf4e0]
data.aws_ami.ubuntu: Read complete after 1s [id=ami-062944a84867f2386]
aws_internet_gateway.igw: Refreshing state... [id=igw-02991fdf487a24963]
aws_subnet.public: Refreshing state... [id=subnet-0486e33a97cf76cd9]
aws_security_group.web: Refreshing state... [id=sg-0aeead241b53fa11e]
aws_route_table.public: Refreshing state... [id=rtb-03688b557599529f3]
aws_instance.app: Refreshing state... [id=i-039214559a9c7bbd9]
aws_route_table_association.public: Refreshing state... [id=rtbassoc-0e297e3830573dc27]
aws_s3_bucket_public_access_block.backup: Refreshing state... [id=library-backup-cb6cf4e0]
aws_s3_bucket_versioning.backup: Refreshing state... [id=library-backup-cb6cf4e0]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

instance_public_ip = "13.220.44.195"
s3_bucket_name = "library-backup-cb6cf4e0"
vpc_id = "vpc-02a21dbf08b1f91f4"
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops/chapter-3/terraform$
```

Figure 5: terraform apply: Apply complete with outputs instance_public_ip, s3_bucket_name and vpc_id

5. Verification of the Provisioned Infrastructure

5.1 Terraform outputs

terraform output prints the values defined in outputs.tf: the public IP of the EC2 instance (13.220.44.195), the S3 bucket name (library-backup-cb6cf4e0) and the VPC ID.

```
terraform output
```

```
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops/chapter-3/terraform$ terraform output
instance_public_ip = "13.220.44.195"
s3_bucket_name = "library-backup-cb6cf4e0"
vpc_id = "vpc-02a21dbf08b1f91f4"
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops/chapter-3/terraform$
```

Figure 6: terraform output: public IP, S3 bucket name and VPC ID

5.2 EC2 instance

The AWS CLI confirms that the instance library-app-server (i-039214559a9c7bbd9) is in the running state with the same public IP.

```
aws ec2 describe-instances --filters "Name=tag:Name,Values=library-app-server" \
  --query "Reservations[].Instances[].[InstanceId,State.Name,PublicIpAddress]" \
  --output table --region us-east-1
```

```
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops/chapter-3/terraform$ aws ec2 describe-instances --filters "Name=tag:Name,Values=library-app-server" --quer
y "Reservations[].Instances[].[InstanceId,State.Name,PublicIpAddress]" --output table --region us-east-1
----------------------------------------------------
|               DescribeInstances                  |
+----------------------+----------+----------------+
|  i-039214559a9c7bbd9 |  running |  13.220.44.195  |
+----------------------+----------+----------------+
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops/chapter-3/terraform$
```

Figure 7: EC2 instance library-app-server is running with public IP 13.220.44.195

5.3 S3 backup bucket

```
aws s3 ls | grep library-backup
```

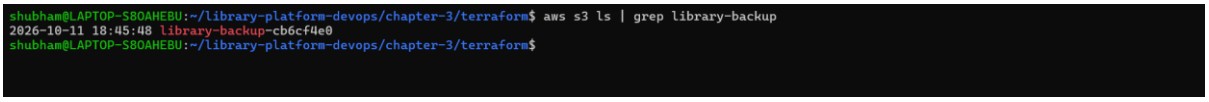


Figure 8: S3 bucket library-backup-cb6cf4e0 created by Terraform

5.4 SSH login to the EC2 instance

The key pair library-aws-key from Chapter 2 is used to log in to the new server. The first connection asks to trust the host fingerprint (answer yes). The Ubuntu 22.04.5 LTS welcome banner confirms a successful login and the private IP 10.0.1.246 matches the subnet 10.0.1.0/24.

```
ssh -i ~/.ssh/library-aws-key ubuntu@$(terraform output -raw instance_public_ip)
```

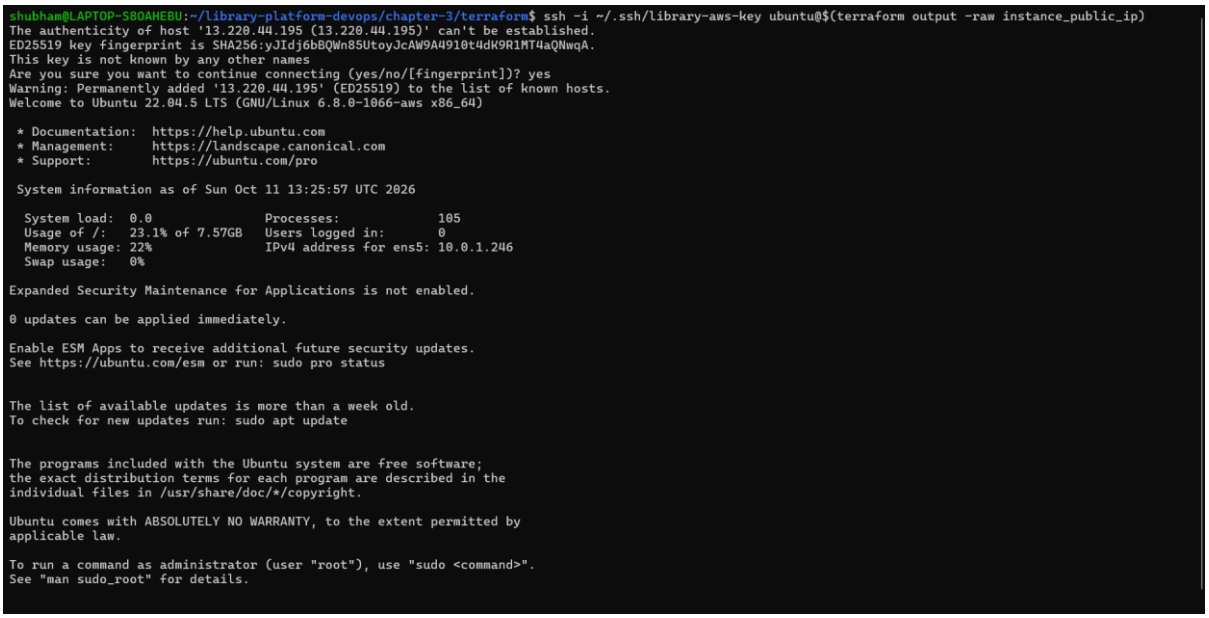


Figure 9: SSH login to the EC2 instance: Ubuntu 22.04.5 LTS, private IP 10.0.1.246

6. Cleanup with terraform destroy

After the verification, all resources are removed so that no EC2 or S3 charges continue. terraform destroy first shows the destroy plan and asks for confirmation.

```
terraform destroy
```

```
shubham@LAPTOP-S8OAHEBU:~/library-platform-devops/chapter-3/terraform$ terraform destroy
random_id.suffix: Refreshing state... [id=y2z04A]
data.aws_ami.ubuntu: Reading...
aws_vpc.main: Refreshing state... [id=vpc-02a21dbf08b1f91f4]
aws_s3_bucket.backup: Refreshing state... [id=library-backup-cb6cf4e0]
data.aws_ami.ubuntu: Read complete after 2s [id=ami-062944a84867f2386]
aws_internet_gateway.igw: Refreshing state... [id=igw-02991fdf487a24963]
aws_subnet.public: Refreshing state... [id=subnet-0486e33a97cf76cd9]
aws_security_group.web: Refreshing state... [id=sg-0aeead241b53fa11e]
aws_route_table.public: Refreshing state... [id=rtb-03688b557599529f3]
aws_instance.app: Refreshing state... [id=i-039214559a9c7bbd9]
aws_route_table_association.public: Refreshing state... [id=rtbassoc-0e297e3830573dc27]
aws_s3_bucket_public_access_block.backup: Refreshing state... [id=library-backup-cb6cf4e0]
aws_s3_bucket_versioning.backup: Refreshing state... [id=library-backup-cb6cf4e0]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

  # aws_instance.app will be destroyed
  - resource "aws_instance" "app" {
      - ami                                  = "ami-062944a84867f2386" -> null
      - arn                                  = "arn:aws:ec2:us-east-1[redacted]:instance/i-039214559a9c7bbd9" -> null
      - associate_public_ip_address          = true -> null
      - availability_zone                    = "us-east-1a" -> null
      - disable_api_stop                     = false -> null
      - disable_api_termination              = false -> null
      - ebs_optimized                        = false -> null
      - force_destroy                        = false -> null
      - get_password_data                    = false -> null
      - hibernation                          = false -> null
      - id                                   = "i-039214559a9c7bbd9" -> null
      - instance_initiated_shutdown_behavior = "stop" -> null
      - instance_state                       = "running" -> null
      - instance_type                        = "t3.micro" -> null
      - ipv6_address_count                   = 0 -> null
      - ipv6_addresses                       = [] -> null
      - key_name                             = "library-aws-key" -> null
      - monitoring                           = false -> null
      - placement_partition_number           = 0 -> null
```

Figure 10: terraform destroy: state is refreshed and the destroy plan starts with aws_instance.app

```
      # (4 unchanged attributes hidden)
    }

  # aws_vpc.main will be destroyed
  - resource "aws_vpc" "main" {
      - arn                                  = "arn:aws:ec2:us-east-1[redacted]:vpc/vpc-02a21dbf08b1f91f4" -> null
      - assign_generated_ipv6_cidr_block     = false -> null
      - cidr_block                           = "10.0.0.0/16" -> null
      - default_network_acl_id               = "acl-0f2d5128d98d1b018" -> null
      - default_route_table_id               = "rtb-0439d5a8d1ddd82f0" -> null
      - default_security_group_id            = "sg-00262d2d962258921" -> null
      - dhcp_options_id                      = "dopt-0b5e863e70eae2be2" -> null
      - enable_dns_hostnames                 = true -> null
      - enable_dns_support                   = true -> null
      - enable_network_address_usage_metrics = false -> null
      - id                                   = "vpc-02a21dbf08b1f91f4" -> null
      - instance_tenancy                     = "default" -> null
      - ipv6_netmask_length                  = 0 -> null
      - main_route_table_id                  = "rtb-0439d5a8d1ddd82f0" -> null
      - owner_id                             = "[redacted]" -> null
      - region                               = "us-east-1" -> null
      - tags                                 = {
          - "Name" = "library-vpc"
        } -> null
      - tags_all                             = {
          - "Name" = "library-vpc"
        } -> null
        # (4 unchanged attributes hidden)
    }

  # random_id.suffix will be destroyed
  - resource "random_id" "suffix" {
      - b64_std     = "y2z04A==" -> null
      - b64_url     = "y2z04A" -> null
      - byte_length = 4 -> null
      - dec         = "3412915424" -> null
      - hex         = "cb6cf4e0" -> null
      - id          = "y2z04A" -> null
    }

Plan: 0 to add, 0 to change, 11 to destroy.
```

Figure 11: Destroy plan summary: Plan: 0 to add, 0 to change, 11 to destroy

After typing yes, Terraform deletes the resources in the correct dependency order: bucket settings and the instance first, then the gateway, subnet and security group, and the VPC last. It ends with "Destroy

complete! Resources: 11 destroyed."

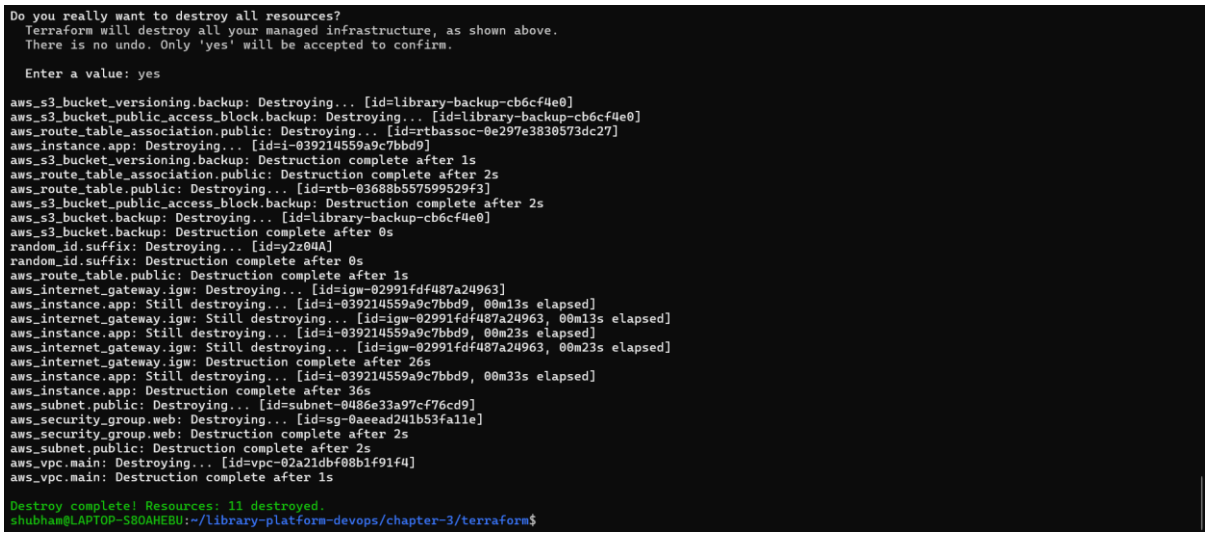


Figure 12: Destroy complete: 11 resources destroyed

> **COST NOTE**
> EC2 instances and S3 storage are billed while they exist. Always run terraform destroy after the lab, and check the AWS Console that no instance or bucket is left behind.

7. GitHub Repository Notes

The Terraform code is stored in chapter-3/terraform of the repository. The state file contains real resource IDs, so it must not be committed.

Commit to Git	Never commit (blocked by .gitignore)
main.tf, variables.tf, providers.tf, outputs.tf	terraform.tfstate and terraform.tfstate.*
.terraform.lock.hcl	.terraform/ directory
chapter-3/screenshots/ (masked screenshots)	*.tfvars, private keys (*.pem, library-aws-key*)

8. Troubleshooting Common Terraform Errors

Symptom / Error Message	Root Cause	Remediation Step
No valid credential sources found	AWS CLI credentials not configured or expired	Run aws configure list and aws sts get-caller-identity, then re-run aws configure if needed.
InvalidKeyPair.NotFound	key_name does not exist in the region	Check aws ec2 describe-key-pairs --region us-east-1 and set the correct key_name.
BucketAlreadyExists	S3 bucket names are global	Keep the random_id suffix so the name is unique.
Error acquiring the state lock	Another terraform command is still running	Wait for it to finish or stop the old process, then retry.

Symptom / Error Message	Root Cause	Remediation Step
Plan shows 11 to add again	State file was lost or deleted	Keep terraform.tfstate in the working folder (but not in Git); do not delete it before destroy.

9. Conclusion

Using Terraform, the complete network and compute foundation of the Enterprise Smart Library Management Platform (VPC, subnet, Internet Gateway, route table, security group, EC2 instance and an S3 backup bucket) was defined as code, validated, applied and verified through Terraform outputs, the AWS CLI and an SSH login. The infrastructure was then removed cleanly with terraform destroy (11 resources). The EC2 instance created here is the target that Ansible will configure with Docker and Nginx in the next chapter.